HƯỚNG DẪN THỰC HÀNH

VoiceScope: CI/CD và Docker cho AI inference

Tài liệu thực hành từng bước | Ngày kiểm chứng: 31/10/2026

Dự án: https://github.com/nqming/voicescope

Bản web hiện có: https://voicescope-581u.onrender.com

Phạm vi: đóng gói và phục vụ inference CPU; không huấn luyện lại model.

Tài liệu phân biệt kết quả đã chạy với hướng dẫn chưa thực thi. Các phần publish Docker Hub và cấu hình credential đang chờ xác nhận theo bộ duyệt tự động. Không có token hay audio riêng tư trong tài liệu.

1. Tổng quan và khảo sát dự án

VoiceScope nhận WAV/FLAC hoặc âm thanh ghi bằng trình duyệt, chuẩn hóa waveform mono 16 kHz và phân loại cấp clip thành real/fake. FastAPI phục vụ cả API và giao diện HTML/CSS/JavaScript tại cùng origin. Audio yêu cầu được xử lý trong RAM; không tạo kho lưu âm thanh ở máy chủ.

Model phục vụ hiện tại là NII AntiDeepfake Wav2Vec2-Small, bản ONNX Q4 với activation FP32. ONNX Runtime 1.20.1 chạy CPU, một luồng tính toán. Không cần CUDA, driver NVIDIA hay NVIDIA Container Toolkit. DINOv3 và các module huấn luyện cũ còn trong mã nghiên cứu nhưng không tham gia model mặc định của image web.

Các endpoint: GET / mở web; GET /health và /api/status trả trạng thái; GET /ready trả 200 khi model sẵn sàng, 503 khi chưa tải được; POST /inference nhận bytes WAV/FLAC, không phải multipart form. GET /docs mở tài liệu OpenAPI.

Python đích: 3.12. Dependencies API được pin trong requirements-runtime.txt. requirements-ci.txt bổ sung pytest và httpx. Dockerfile có nhiều stage: runtime API, inference DINOv3 tùy chọn cũ, antideepfake-build để chuyển đổi weights và web là image inference mặc định cuối cùng. Compose local đã chỉ rõ target web.

Weights Q4 được lưu tại /opt/models/antideepfake/model_q4.onnx. Revision upstream cố định: 9a13264b5dcc827a8d5a4f8e01fccefa392f886b. Kích thước và checksum thực tế có trong báo cáo image audit. Build tải public checkpoint, kiểm checksum nguồn, chuyển đổi và đóng gói weights. Runtime không cần tải model qua mạng.

Không tìm thấy AGENTS.md áp dụng trong thư mục dự án và các thư mục cha đã kiểm tra. Repository root có thay đổi sẵn của người dùng; việc chuẩn bị commit CI dùng checkout riêng tmp/cicd-release, không reset hay commit lẫn dữ liệu của bạn. Đường dẫn hiện tại là D:\Codex\BTTHCDTTNT; đường dẫn Bai2_CD trong lịch sử không còn tồn tại.

2. CI, CD, image, container và luồng triển khai



PR dừng sau audit. Publish và deploy là hai bước riêng.

CI là các kiểm tra tự động khi code thay đổi: test Python, test ghi âm PCM, build image và gọi inference thật trong container. CD là bước đưa một image đã kiểm chứng lên môi trường chạy và kiểm tra sau triển khai.

Docker image là gói bất biến chứa chương trình, dependencies và model. Container là một phiên chạy của image. Inference là đưa audio qua model để lấy điểm và nhãn, không cập nhật weights. Tín hiệu sin tổng hợp trong smoke test chỉ kiểm đường thực thi; không phải mẫu dùng chứng minh độ chính xác.

Bản Render hiện tại dùng source-based Docker build và auto-deploy tắt. Việc publish image lên Docker Hub không đồng nghĩa image đó đã được deploy lên Render. Không tạo server hoặc hạ tầng trả phí trong công việc này.

3. Điều kiện chuẩn bị và cấu trúc file

Mục đích: có môi trường đủ để tái lập build, kiểm thử và vận hành.

Cần Git, Docker Desktop dùng Linux containers, Python 3.12 và Node.js 22 cho test JavaScript. Docker Engine kiểm tra thực tế trên máy này là 29.9.2, kiến trúc linux/amd64. Cần tài khoản GitHub có quyền ghi repository và Docker Hub có quyền push repository đích. Không cần sounddevice, dataset hay microphone để chạy CI.

Build cần mạng đến PyPI, PyTorch CPU index và Hugging Face. Nên dành ít nhất 8 GB RAM và 10 GB trống cho builder vì stage chuyển đổi có PyTorch và weights FP32. Đây là dự trù vận hành, không phải đo đạc RAM tối thiểu. Image cuối không chứa PyTorch.

Từ thư mục gốc dự án trong PowerShell:

```
Set-Location D:\Codex\BTTHCDTTNT
docker version
docker context show
python --version
node --version
```

Không có giá trị cần thay nếu dùng đúng máy hiện tại. Máy khác thay đường dẫn Set-Location bằng thư mục clone của bạn. Kết quả mong đợi: Docker hiển thị cả Client và Server, context desktop-linux, Python 3.12.

Các file chính:

- Dockerfile, .dockerignore: base digest, stage model và allowlist context.
- MODEL-NOTICE.md, licenses/: nguồn, thay đổi model và license gốc.
- requirements-runtime.txt, requirements-ci.txt: dependencies API và test.
- compose.yaml: build và chạy local; compose.production.yaml: chạy image theo digest.
- tools/container_smoke.py: kiểm readiness và thực thi inference qua HTTP.
- tools/audit_image.py: kiểm model checksum, payload và user runtime.
- tools/publish_image.py: tag và push image đã test.
- .github/workflows/ci-docker.yml: CI và publish có điều kiện.
- .github/workflows/deploy-docker-host.yml, deploy/remote.sh: CD riêng.
- voicescope/smoke.py, tests/test_api.py: cập nhật giả định UI/API cũ.

4. Chạy ứng dụng và test local

Mục đích: xác minh mã API và pipeline cơ bản trước khi build. File dùng: requirements-ci.txt và tests/. Thư mục chạy: gốc repository.

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements-ci.txt
.\.venv\Scripts\python.exe -m pytest -q
node tests/test_pcm_capture.cjs
.\.venv\Scripts\python.exe -m voicescope.server
```

Nếu đã có .venv thì dùng môi trường hiện có, không cần tạo lại. Mở http://127.0.0.1:8000. API local không tự có model chỉ nhờ cài dependencies; model đã đóng gói sẵn trong Docker là cách khuyến nghị cho người mới.

Nếu đã có bộ ONNX hợp lệ và muốn chạy native, cài onnxruntime==1.20.1 rồi đặt:

```
$env:VOICESCOPE_MODEL = 'antideepfake-int8'
$env:VOICESCOPE_ANTIDEEPFAKE_DIR = 'tmp/antideepfake-q4-deploy'
.\.venv\Scripts\python.exe -m voicescope.server
```

Tên selector antideepfake-int8 được giữ để tương thích cấu hình cũ; weights thực tế là Q4, không phải INT8. Thay đường dẫn model bằng thư mục chứa model_q4.onnx và manifest.json khớp. Không dùng checkpoint bất kỳ hoặc sửa manifest để vượt qua kiểm tra checksum.

Kết quả kiểm tra mong đợi: test pass; /health có model_ready phù hợp. Ứng dụng chưa có model phải trả 503 cho /inference thay vì bịa điểm. Khi dừng native server dùng Ctrl+C. Không chạy hai server cùng cổng 8000.

5. Build image và kiểm tra nội dung

Mục đích: tạo gói inference tự chứa và tránh đưa dữ liệu riêng vào Docker. File: Dockerfile, .dockerignore, tools/export_antideepfake.py, MODEL-NOTICE.md và licenses/. Chạy tại gốc repository.

```
$revision = git rev-parse HEAD
docker build --target web --build-arg VCS_REF=$revision `
  --tag voicescope:cicd-test --progress plain .
python tools/audit_image.py --image voicescope:cicd-test
docker image inspect voicescope:cicd-test `
  --format '{{.Config.User}} {{.Size}}'
```

Trong workspace đã chuyển đường dẫn, nếu Git báo không có work tree, dùng git --git-dir=.git --work-tree=. rev-parse HEAD hoặc dùng checkout tmp/cicd-release. VCS_REF phải là commit của source bạn thực sự build. Build workspace còn thay đổi chưa commit dùng nhãn workspace-cicd để tránh giả là một commit sạch.

Base Python được pin digest trong Dockerfile. API dependencies được pin exact version; các thư viện ML chính của builder cũng được pin. Dependency bắc cầu của builder chưa có lockfile kèm hash đầy đủ, vì vậy không khẳng định rebuild khác thời điểm sẽ giống từng byte. Image digest đã publish là cách tái dùng đúng gói đã test.

Allowlist context chỉ cho Dockerfile, requirements-runtime.txt, source .py, web HTML/CSS/JS, config mặc định, exporter, notice và license. Không đưa .git, .env, credential, data/, tmp/, artifacts/, notebook, PDF hay audio vào context. Final stage không sao chép /opt/source hay môi trường PyTorch của builder.

Model dùng giấy phép CC BY-NC-SA 4.0. Notice ghi rõ tác giả upstream, revision, thay đổi ONNX/Q4 và license của weights phái sinh. Model card, văn bản license, manifest và checksum được giữ trong image. Public image phục vụ nghiên cứu/giáo dục phi thương mại; public không có nghĩa được dùng thương mại.

Audit phải xác nhận UID khác 0, không có torch trong runtime, không có audio/secret trong payload ứng dụng và SHA256 weights đúng manifest. Đây là kiểm tra payload dự án; chưa phải chứng nhận image không có mọi lỗ hổng bảo mật.

6. Chạy container và gọi API dự đoán

Mục đích: kiểm ứng dụng thực sự tải và chạy model. File: tools/container_smoke.py. Chạy tại gốc repository, Docker Desktop đang hoạt động.

```
docker run -d --name voicescope-cicd-test `
  --read-only --tmpfs /tmp:size=64m,mode=1777 `
  --cap-drop ALL --security-opt no-new-privileges `
  -p 127.0.0.1:8000:8000 voicescope:cicd-test
python tools/container_smoke.py
Invoke-RestMethod http://127.0.0.1:8000/ready
docker logs voicescope-cicd-test
```

Tên container có thể thay nếu bị trùng. Nếu cổng bận, đổi phần trước dấu hai chấm thành 8001 và truyền --url http://127.0.0.1:8001 cho smoke. Kết quả mong đợi: /ready trả model_ready=true; smoke ghi artifacts/container-smoke.json với nhãn và fake_score hữu hạn. Smoke tự tạo WAV sin 2 giây trong RAM, chỉ xác minh khả năng thực thi.

Gọi inference bằng một WAV/FLAC của bạn được phép xử lý:

```
curl.exe --fail-with-body -X POST `
  -H 'Content-Type: audio/wav' `
  --data-binary '@sample.wav' `
  http://127.0.0.1:8000/inference
```

Thay sample.wav bằng tệp thật; với FLAC dùng Content-Type: audio/flac. Không dùng -F vì endpoint nhận raw body. Chạy HTTP local chỉ giới hạn loopback; ghi âm trong trình duyệt ở môi trường public cần HTTPS.

Dừng và xóa đúng container thử nghiệm sau khi kiểm tra:

```
docker stop voicescope-cicd-test
docker rm voicescope-cicd-test
```

Lệnh này chỉ xóa container thử, không xóa image hay dữ liệu trên máy. Có thể dùng docker compose up --build -d cho lần chạy local tiếp theo; compose.yaml chọn target web và giới hạn cổng loopback.

7. Docker Hub và secrets CI

Mục đích: cấp khả năng push có kiểm soát. File workflow: .github/workflows/ci-docker.yml. Repository đích dự kiến: qminh19/voicescope; namespace được xác định từ credential helper Docker đang đăng nhập.

Trang đích dự kiến: https://hub.docker.com/r/qminh19/voicescope. Tại thời điểm viết, repository chưa được tạo và URL này chưa phải bằng chứng publish thành công. Bộ duyệt tự động đã từ chối thao tác tạo repository public và chuyển PAT đang lưu sang GitHub secret; cần người dùng xác nhận cụ thể trước khi tiếp tục thao tác đó.

Trong Docker Hub, tạo repository tên voicescope thuộc qminh19, visibility Public. Dùng PAT chỉ có quyền cần thiết cho push. Trong GitHub vào repository > Settings > Secrets and variables > Actions, tạo:

- Variable DOCKERHUB_USERNAME: qminh19.
- Secret DOCKERHUB_TOKEN: PAT Docker Hub có quyền push.

Không dán PAT vào chat, source, log hoặc tài liệu. Nhập PAT vào giao diện secrets chính thức hoặc GitHub CLI tương tác an toàn. Để đăng nhập local nếu cần, dùng docker login --username qminh19, nhập token ở prompt bảo mật. Không đặt --password TOKEN trên dòng lệnh.

```
docker login --username qminh19
```

Chạy từ bất kỳ thư mục nào. Chỉ nhập credential khi chương trình yêu cầu; không thay username bằng email. Kiểm tra thành công khi Docker báo Login Succeeded và tên secret xuất hiện trong GitHub; không mở giá trị secret để xác minh.

PR dùng event pull_request và quyền contents: read. Không dùng pull_request_target để chạy code PR có secrets. Bước login/push chỉ chạy sau các kiểm tra và khi event là push trên main, release/** hoặc version tag. Action bên thứ ba được pin commit SHA từ repository chính thức.

8. Kích hoạt CI và quy tắc tag

Mục đích: kiểm code tự động và truy vết image về commit. File: .github/workflows/ci-docker.yml và tools/publish_image.py. Chạy Git từ checkout dùng cho CI; không add toàn bộ workspace đang có dữ liệu/thay đổi chưa kiểm tra.

Lịch sử tích hợp: PR #2 đã được thay thế; PR #3 (https://github.com/nqming/voicescope/pull/3) đã merge mã hiện tại vào main. Nhánh ci/voicescope-docker ban đầu tách từ feature/deploy-voicescope. Khi cập nhật PR, tests và image job tự chạy. Mở tab Checks hoặc Actions, đọc log bước thất bại; báo cáo JSON nằm trong artifact container-evidence.

```
git switch -c feature/my-change
# Sua code va commit cac file lien quan
git push -u origin feature/my-change
```

Thay tên nhánh bằng tên việc của bạn. Tạo PR vào nhánh đã chứa workflow. Kết quả mong đợi: tests pass rồi image build và inference pass. PR không push image.

Sau khi merge phần CI vào nhánh phát hành được tin cậy và cấu hình credentials, push main hoặc release/** tạo tag sha-<40-ky-tu-commit>. Tag này truy vết source; không cập nhật latest ở mỗi commit nhánh.

Ví dụ phát hành một phiên bản ổn định chưa tồn tại:

```
git tag v0.2.0
git push origin v0.2.0
```

Chỉ chạy sau kiểm tra và dùng phiên bản chưa có. Tag vMAJOR.MINOR.PATCH tạo tag cùng tên và latest; prerelease như v0.2.0-rc.1 không cập nhật latest. Workflow vẫn thêm tag SHA đầy đủ. Chưa tạo version tag trong lần thực hiện này. Không ghi đè tag phát hành cũ; sửa lỗi bằng phiên bản mới. Deploy production nên dùng digest, không dùng latest để rollback.

9. Push thủ công, pull công khai và chạy image

Mục đích: công bố đúng image đã test, xác minh người không đăng nhập cũng lấy được image. File: tools/publish_image.py. Chạy tại gốc source đã build; cần credential publish được phê duyệt trước.

```
$env:GITHUB_SHA = git rev-parse HEAD
$env:GITHUB_REF = 'refs/heads/release/cicd-docker'
$env:DOCKERHUB_USERNAME = 'qminh19'
python tools/publish_image.py --image voicescope:cicd-test
```

GITHUB_SHA phải đúng source revision của image; không gán commit mới cho image cũ. Script thêm sha-<SHA>, tag image rồi push, không build lại. Kiểm tra artifacts/publish.json để lấy digest; chỉ báo đã publish khi lệnh push thực sự thành công.

Sau publish, dùng Docker config tạm trống để thử pull ẩn danh thay vì logout phiên của bạn:

```
New-Item -ItemType Directory tmp/docker-anonymous -Force
# Thay <TAG_DA_PUBLISH> bang tag da xac minh
docker --config tmp/docker-anonymous pull `
  qminh19/voicescope:<TAG_DA_PUBLISH>
```

Không có tag hoặc digest Docker Hub đã xác minh tại thời điểm tài liệu này. Những chỗ trong dấu <...> là giá trị phải lấy từ lần push thành công, không phải digest giả. Khi có digest, dùng:

```
docker pull qminh19/voicescope@sha256:<DIGEST_THUC_TE>
docker run -d --name voicescope `
  --read-only --tmpfs /tmp:size=64m,mode=1777 `
  --cap-drop ALL --security-opt no-new-privileges `
  -p 127.0.0.1:8000:8000 `
  qminh19/voicescope@sha256:<DIGEST_THUC_TE>
```

Sau đó chạy smoke ở mục 6. Kiến trúc image trong pipeline này là linux/amd64; không tuyên bố đã thử ARM64. Lệnh pull chỉ được xem là kiểm chứng công khai sau khi dùng config trống và nhận manifest/layers thành công.

10. Deploy và rollback

Mục đích: đưa image đã test lên Docker host hiện có. File: compose.production.yaml, deploy/remote.sh và workflow deploy-docker-host.yml. Chưa có Docker host SSH được cấu hình; workflow này là cấu hình triển khai mẫu, chưa thực thi trên server.

Trên Linux host có Docker Compose và Python 3, clone source tin cậy chứa các file trên vào đường dẫn tuyệt đối, ví dụ /srv/voicescope. Cấu hình HTTPS reverse proxy trỏ về 127.0.0.1:8000; đặt VOICESCOPE_ALLOWED_HOSTS chứa hostname thật. Không expose cổng Docker trực tiếp ra Internet trong cấu hình mẫu này.

```
cd /srv/voicescope
export VOICESCOPE_ALLOWED_HOSTS=voicescope.example.com,localhost,127.0.0.1
bash deploy/remote.sh qminh19/voicescope@sha256:<DIGEST_THUC_TE>
```

Thay domain, thư mục và digest bằng giá trị thật. Script pull trước, chạy compose --wait, gọi readiness và inference rồi lưu digest hiện tại/trước đó trong .deploy-state/. Nếu kiểm tra thất bại và có digest trước đó, script khôi phục image trước. Lần deploy đầu chưa có bản cũ để rollback.

Rollback thủ công trên cùng host:

```
cd /srv/voicescope
bash deploy/remote.sh "$(cat .deploy-state/previous)"
```

Trước tiên kiểm tra previous không rỗng. Rollback cũng phải qua health và inference. Muốn kích hoạt qua Actions, tạo environment production và cấu hình người review trước khi deploy nếu tài khoản hỗ trợ; thêm vars DEPLOY_HOST, DEPLOY_USER, DEPLOY_PATH và secrets DEPLOY_SSH_KEY, DEPLOY_KNOWN_HOSTS. KNOWN_HOSTS phải được xác minh qua kênh tin cậy; không tự tin kết quả ssh-keyscan chưa xác nhận. Host checkout phải được cập nhật từ nguồn đã review, workflow không tự chạy git pull tùy ý.

Bản Render hiện tại dùng repository, không nhận image từ workflow này. Chưa có phiên/API Render dùng được trong lượt này để đổi cơ chế deploy và kiểm tra rollback. Để dùng Docker Hub trên Render cần cấu hình service lấy external image, chọn digest đã publish, giữ PORT/health phù hợp, deploy và kiểm tra URL; việc chuyển service có thể thay đổi cấu hình nên chưa thực hiện. Không tạo thêm service trả phí. Bản web hiện có không phải bằng chứng cho CD mới đã chạy.

11. Lỗi đã gặp và cách xử lý

Docker API permission denied: shell sandbox không truy cập được named pipe Docker Desktop. Kiểm tra Engine ở phiên có quyền phù hợp; không sửa bảo mật pipe toàn hệ thống. Docker Desktop phải đang chạy Linux engine.

Directory name invalid: workspace cũ D:\Codex\Bai2_CD không tồn tại. Dùng đường dẫn thực tế D:\Codex\BTTHCDTTNT. Không tạo dự án mới vào đường dẫn cũ hoặc ghi đè dữ liệu.

Git báo must be run in a work tree: trong môi trường này, dùng checkout độc lập tmp/cicd-release hoặc chỉ rõ --git-dir và --work-tree khi cần. Không xóa .git hay reset thay đổi để xử lý.

HTTP 403 khi tải legalcode.en.txt: lần build đầu đã tải và export weights, nhưng thất bại ở bước tải văn bản license. Đã thay bằng license gốc từ repository Creative Commons, lưu nguồn commit và checksum tại licenses/SOURCE.md. Image không còn phụ thuộc request đó khi build.

Model chưa ready: /health có thể trả 200 trong khi model_ready=false. Dùng /ready để kiểm model và smoke để thực thi thật. Kiểm manifest, checksum, revision và log Docker. Không đổi nhãn/threshold để làm test qua.

Port/container name đã dùng: chọn cổng/name khác hoặc dừng đúng container thử của mình. 415: gửi WAV/FLAC raw body với Content-Type đúng. 422: audio không decode được, quá ngắn, quá im lặng hoặc vi phạm giới hạn. 429: server đang bận hoặc vượt rate limit; thử lại sau.

Lỗi bash unexpected end of file khi kiểm qua pipe Windows: gửi bytes LF thay vì để subprocess chuyển input thành CRLF; .gitattributes giữ script .sh là LF trong checkout Linux.

CI lần đầu thiếu test_pcm_capture.cjs trên nhánh deploy: đã bổ sung tệp test từ workspace. Audit lần đầu còn thấy __pycache__: đã siết lại exclude bên trong từng thư mục và build lại, không chỉ bỏ qua lỗi audit.

Push unauthorized hoặc insufficient_scope: xác minh namespace, repository và quyền PAT. Không in token ra log. Actions không có secret: cấu hình đúng tên, event PR không được publish. Q4 yêu cầu ORT tương thích; bản đã chọn là 1.20.1.

TestClient có cảnh báo deprecation httpx trên phiên Starlette hiện tại; bộ test vẫn pass. Cần theo dõi nâng cấp tương thích riêng, không tự bỏ test để che cảnh báo.

12. Kết quả kiểm chứng thực tế

CI main mới nhất đã kiểm: THÀNH CÔNG, https://github.com/nqming/voicescope/actions/runs/41160259928. Test Python, test PCM, build Linux/amd64, ONNX inference và audit image đều pass. Login/publish được SKIP vì chưa cấu hình Docker Hub.

- Source build: checkout sạch, commit cf09394c83097a5e2c0f6290ac144e795fdfa144; giữ nguyên README người dùng vừa sửa. OCI label và /health.version khớp commit.
- Tag local đã kiểm: voicescope:sha-cf09394c83097a5e2c0f6290ac144e795fdfa144; dung lượng 833.850.413 bytes, khoảng 795 MiB.
- ID local: sha256:86beb401656b893088467a32f5d49af11bec0d9c331e125c7f0271c62f2fbc8f. Đây KHÔNG phải digest registry đã publish.
- Container chạy UID 10001, filesystem read-only, tmpfs /tmp, drop capabilities và no-new-privileges. Model ready, health và các endpoint HTTP pass.
- Inference sin 2 giây chạy được. Câu Windows TTS 5,368 giây: fake, fake_score 0,99997365; expected-label=fake pass. Đây là kiểm thực thi, không phải benchmark accuracy.
- Payload sạch: không có torch, audio, token hoặc __pycache__ trong /app; có model card, notice, license và manifest. Weights 89.730.755 bytes, SHA256 66a094e0d039c899e07fd08f8c3ff6ecf24145ce8bf5f53a3d23e25c5775f4f5 khớp manifest.
- Kiểm tra trước đó: local 25 passed/1 skipped; CI Linux 23 passed/2 skipped; test PCM pass. Compose và bash -n hợp lệ; chưa chạy remote deploy.

Chạy đúng image đã có trên máy, không cần Docker Hub:

```
docker run --rm -p 127.0.0.1:8000:8000 `
  voicescope:sha-cf09394c83097a5e2c0f6290ac144e795fdfa144
```

Các lệnh voicescope:cicd-test ở phần hướng dẫn là tag local mẫu. Image theo SHA trên đây thay thế bản thử build từ workspace chưa commit. Log/JSON ở artifacts/; câu TTS ở tmp/; các thư mục này bị loại khỏi Git/image. Không dùng bản ghi microphone riêng của người dùng.

Publish Docker Hub: CHƯA THỰC HIỆN, chưa có pull công khai hay registry digest được xác minh. Auto-review yêu cầu xác nhận cụ thể trước khi chuyển PAT đang lưu sang secret GitHub Actions và tạo public qminh19/voicescope. Lệnh bị chặn chưa thực thi; chưa có credential được chuyển.

CD: đã có cấu hình deploy/rollback nhưng CHƯA deploy image mới. Bản Render giữ nguyên; lần kiểm read-only trước trả /health và /ready 200, revision f196402. Không tạo hạ tầng trả phí.

13. Giới hạn và việc cần thêm cho production

Model có thể false positive/false negative; chưa có bằng chứng đủ để bảo đảm mọi giọng nói/ngôn ngữ/thiết bị. Test sin và vài mẫu benchmark chỉ kiểm thực thi và regression. Cần tập held-out đủ lớn, calibration, đo chất lượng theo thiết bị/ngôn ngữ và kiểm drift.

Cần bổ sung quét CVE/SBOM và ký image theo chính sách vận hành; lockfile kèm hash cho toàn bộ dependency bắc cầu; quy trình cập nhật base digest và action SHA. Chưa thực hiện kiểm thử tải, đo RAM peak hay xác minh GPU/ARM64.

API hiện dùng một worker và giới hạn request theo process; production cần gateway/rate limit chung, TLS, auth nếu phù hợp, quan sát lỗi và hiệu năng không log audio, giới hạn tài nguyên và timeout tại reverse proxy. Không coi liveness là chất lượng model. Không dùng kết quả như kết luận giám định.

Việc đang chờ: phê duyệt cụ thể để tạo repository Docker Hub public và lưu PAT vào GitHub Actions; sau đó push image, kiểm pull ẩn danh, ghi tag/digest thật, kích hoạt publish job và cập nhật kết quả tài liệu. CD server cần host SSH đã có hoặc quyền cấu hình Render tương ứng. Không yêu cầu gửi token vào chat.

Nguồn kỹ thuật chính thức để tra cứu thêm:

- Docker GitHub Actions: https://docs.docker.com/build/ci/github-actions/
- GitHub secrets: https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-secrets
- Docker image digest: https://docs.docker.com/reference/cli/docker/image/pull/
- NII model và giấy phép: https://huggingface.co/nii-yamagishilab/wav2vec-small-anti-deepfake
- Creative Commons: https://creativecommons.org/licenses/by-nc-sa/4.0/